



1. Problem Title & Link

Title: LeetCode 791 : Custom Sort String

Link: [LeetCode 791 - Custom Sort String](#)

2. Problem Statement (Short Summary)

You are given two strings:

order

s

order contains a custom character order.

Rearrange characters in s such that:

- Characters appearing in order follow the same order
- Characters not present in order can appear anywhere

In simple terms:

Sort s according to the custom sequence given in order.

3. Examples (Input → Output)

Example 1

Input:

order="cba"

s="abcd"

Output:

"cbad"

Explanation:

Custom order:

c → b → a

Remaining:

d

can be placed anywhere.

Example 2

Input:

order="bcafg"

s="abcd"

Output:

"bcad"



Explanation:

Characters b,c,a follow custom order.

Example 3

Input:

order="kqep"

s="pekeq"

Output:

"kqeeep"

4. Constraints

$1 \leq \text{order.length} \leq 26$

$1 \leq \text{s.length} \leq 200$

order and s contain lowercase English letters

Special restrictions:

- Characters in order are unique
- Need efficient sorting

5. Core Concept (Pattern / Topic)

HashMap / Frequency Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every character in order:

- Search entire s
- Append matching characters repeatedly

Time:

$O(m*n)$

Repeated scanning is expensive.

Optimized Thinking

Observation:

We only need frequency information.

Algorithm:

1. Count frequency of every character in s



2. Traverse order
3. Append characters according to their count
4. Append remaining characters not in order

Why it works:

Frequency stores occurrences

order controls placement

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

order="cba"

s="abcd"

Frequency:

a → 1

b → 1

c → 1

d → 1

Process:

Take c → "c"

Take b → "cb"

Take a → "cba"

Remaining:

d → "cbad"

8. Pseudocode (Language Independent)

create frequency[26]

count characters in s

result=""



for char in order:

 append char frequency times

 frequency[char]=0

for remaining chars:

 append frequency times

return result

9. Code Implementation

Python

class Solution:

```
def customSortString(self,  
    order,  
    s):
```

```
    freq=[0]*26
```

```
    for c in s:  
        freq[ord(c)-ord('a')] +=1
```

```
    result=[]
```

```
    for c in order:
```

```
        index=ord(c)-ord('a')
```

```
        while freq[index]>0:
```

```
            result.append(c)
```

```
            freq[index]-=1
```



```
for i in range(26):

    while freq[i]>0:

        result.append(
            chr(i+ord('a'))
        )

        freq[i]-=1

    return "".join(result)
```

Java

```
class Solution {

    public String customSortString(
        String order,
        String s) {

        int[] freq=new int[26];

        for(char c:s.toCharArray())
            freq[c-'a']++;

        StringBuilder result=
            new StringBuilder();

        for(char c:
            order.toCharArray()){

            int index=c-'a';

            while(freq[index]>0){

                result.append(c);
```



```
        freq[index]--;
    }
}

for(int i=0;i<26;i++){

    while(freq[i]>0){

        result.append(
            (char)(i+'a')
        );

        freq[i]--;
    }
}

return result.toString();
}
}
```

10. Time & Space Complexity

Time Complexity:

$O(n+m)$

Where:

$n = s.length$

$m = order.length$

Reason:

- Frequency counting $\rightarrow O(n)$
- Traversing order $\rightarrow O(m)$

Space Complexity:

$O(1)$

Reason:

Frequency array size always remains:



26

11. Common Mistakes / Edge Cases

1. Forgetting remaining characters

Wrong:

```
order="cba"
```

```
s="abcd"
```

Output:

```
"cba"
```

Correct:

```
"cbad"
```

2. Using sorting directly

Regular sorting ignores custom order.

3. Forgetting duplicate characters

Input:

```
order="cba"
```

```
s="aabbcc"
```

Output:

```
ccbbaa
```

4. Missing single character case

```
order="a"
```

```
s="a"
```

12. Detailed Dry Run (Step-by-Step Table)

Input:

```
order="cba"
```

```
s="abcd"
```

Frequency:



a	1
b	1
c	1
d	1

Processing:

Step	Result
Take c	"c"
Take b	"cb"
Take a	"cba"
Add remaining d	"cbad"

Final:

cbad

13. Common Use Cases (Real-Life / Interview)

Custom ranking systems

Priority-based sorting

Search result ordering

Personalized recommendation systems

14. Common Traps (Important!)

1. Forgetting leftover characters
2. Using nested loops unnecessarily
3. Sorting entire string directly
4. Missing repeated character frequencies

15. Builds To (Related LeetCode Problems)

Progression:

- LC 791 → Custom Sort String
- LC 49 → Group Anagrams
- LC 451 → Sort Characters By Frequency
- LC 692 → Top K Frequent Words
- LC 347 → Top K Frequent Elements

16. Alternate Approaches + Comparison



Nested Scan	$O(m*n)$	$O(1)$	Repeated searching
HashMap + Sort	$O(n \log n)$	$O(n)$	Sorting overhead
Frequency Array	$O(n+m)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

The frequency array stores how many times each character appears, while order determines where those characters should be placed. Remaining characters are added afterward, ensuring correct custom ordering.

18. Variations / Follow-Up Questions

1. Support uppercase letters
2. Support Unicode characters
3. Sort words instead of characters
4. Sort by multiple custom priorities
5. Build a custom comparator solution
- 6.